

BUILDING A VISION LAYER FOR A REAL FARM

Computer vision, structured farm state, harvest recovery, and forecasting on a 76-site hydroponic rack

What worked, what failed, and what I would keep in the next version



A fixed camera view became an operational measurement stream rather than just a photo archive.

45 analyzed snapshots final archive	24 harvest records linked production history	1.6 d measured recovery after 109 g cut	69 active plants closeout state
---	---	---	---------------------------------------

The short version

The farm began as a physical hydroponics experiment, but the instrumentation became a second engineering project: could a fixed camera, a few sensors, and structured records turn a rack of plants into a measurable system?

I built a local Python/OpenCV pipeline that aligned rack images, isolated the useful crop region, measured green canopy, generated overlays, and fed a Streamlit dashboard. In parallel, I built a structured site ledger for occupancy, harvest records, farm events, environmental history, and forecast resolution.

Bottom line

Computer vision became useful as a trend sensor: it could show canopy growth, quantify the immediate effect of a harvest, and estimate recovery. It was much less trustworthy as an exact per-plant inventory system. The final architecture therefore kept exact farm state deterministic and used vision as supporting evidence rather than the source of truth.

1. The technical question

The question was not whether a model could identify basil in a photo. It was operational: can an inexpensive local vision layer reduce uncertainty about what is growing, how quickly the canopy is changing, what a harvest did to the system, and when the rack has recovered?

- Use a fixed camera and commodity edge hardware rather than a specialized imaging rig.
- Keep the pipeline local and understandable: capture, align, crop, segment, measure, overlay, log.
- Tie image-derived trends to real farm events and measured harvest weights.
- Know when not to trust vision: exact occupancy and cultivar identity must remain ledger-backed.

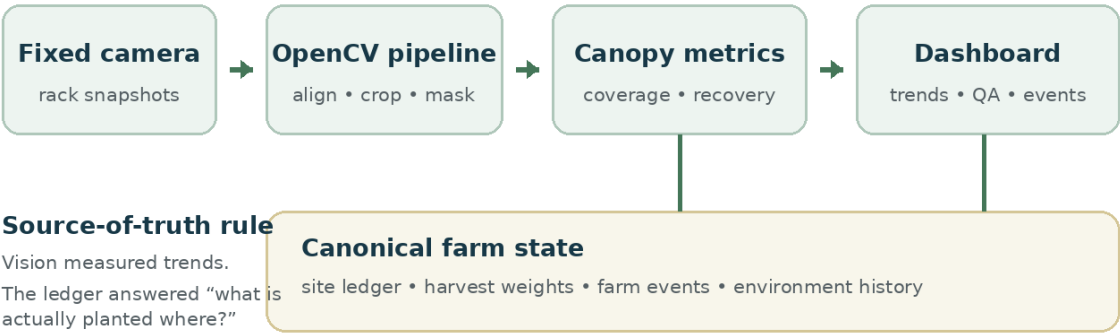
The original June dashboard was a mid-project UI checkpoint. For this case study, figures were rebuilt from the frozen September repository archive and reviewed experiment workbook wherever possible. The June report remains only as a historical reference for how the interface looked at the time.

2. The measurement stack

The useful architecture was deliberately simple. A camera supplied repeatable observations; OpenCV converted them into trend metrics; the dashboard made those metrics legible; and structured records held the facts that vision could not reliably infer.

How the measurement stack evolved

Keep perception useful, but keep farm state deterministic.



The implemented image path: Capture → Align → Crop → Mask → Overlay → Dashboard.

Deployed in the farm, not staged in a lab



Rack deployment: the camera/network hardware shared the same environment as grow lights, humidity, foliage, and changing geometry. The purple cultivar also exposed the limits of green-pixel segmentation.

That physical deployment shaped the software architecture. Wi-Fi interruptions, camera movement, grow-light glare, occlusion, and cultivar color variation made confidence and provenance as important as segmentation. The system therefore evolved toward explicit alignment checks, calibration epochs, sensor-freshness warnings, and a structured ledger that remained authoritative when the image pipeline was unavailable or ambiguous.

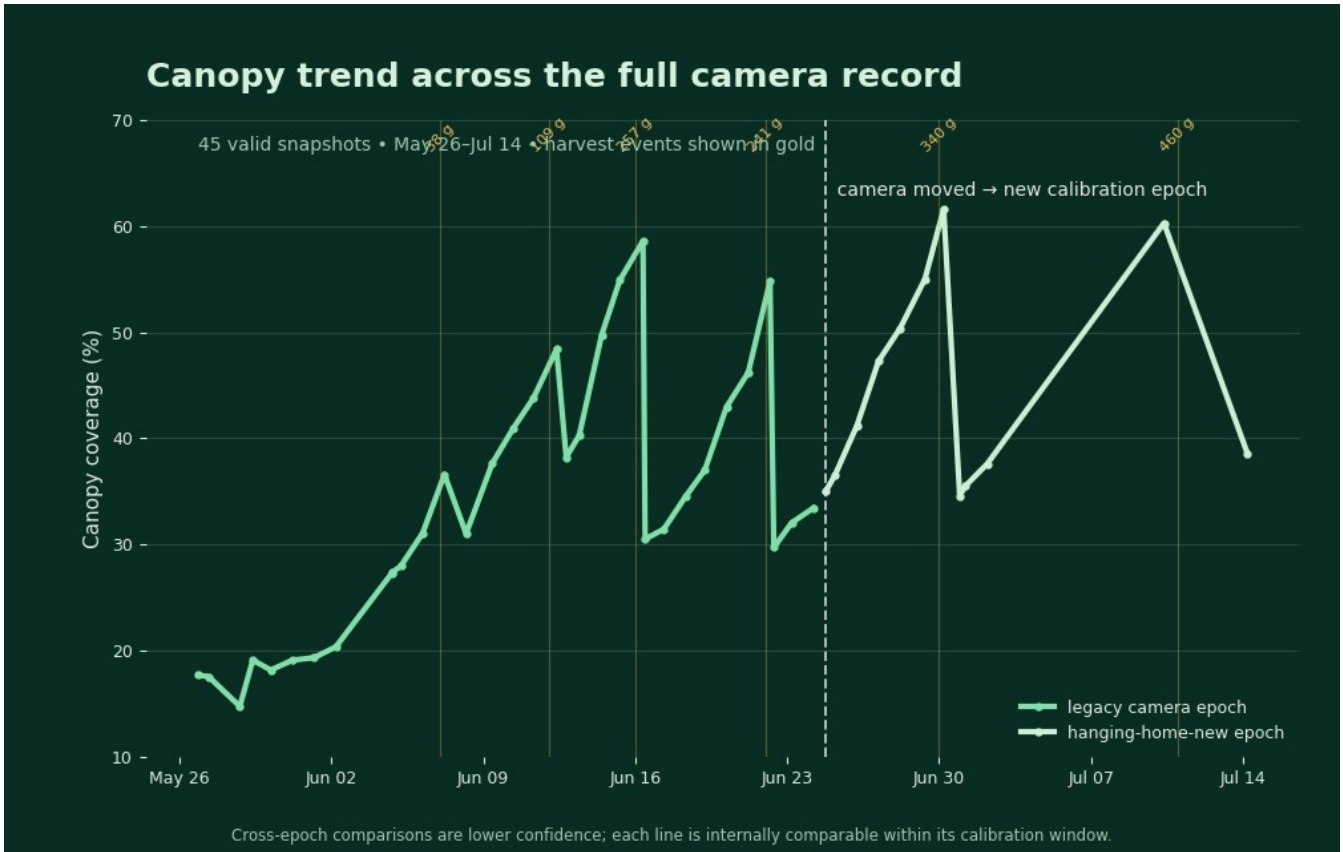
A source-of-truth hierarchy emerged

Data source	Best use	Authority
Site ledger	occupancy, cultivar, movements	canonical
Harvest weights	production and yield	canonical
Farm events	interventions and operating history	canonical
Environment log	temperature / humidity context	measured, gap-aware
Vision metrics	canopy trend, recovery, visual corroboration	supporting / experimental

The key design decision was to stop asking the camera to be a database.

3. Canopy vision worked best as a trend sensor

Whole-rack canopy coverage turned out to be the highest-value vision metric. It did not need to know which leaf belonged to which plant. It only needed a stable frame, a consistent crop, and enough segmentation quality to compare the rack with itself over time.

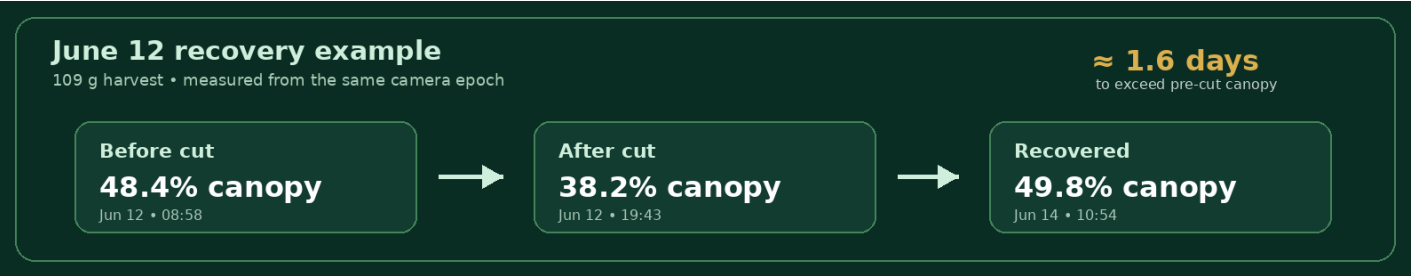


Full camera record: 45 valid snapshots across two calibration epochs, with major harvests visible as deliberate discontinuities in the canopy trend.

Why this mattered

A growth curve turns “the plants look bigger” into a measurable operating signal. More importantly, harvest markers make the curve useful for production: the system can distinguish normal growth from a deliberate biomass removal and then observe the rebound.

The 109 g harvest gave a concrete recovery measurement

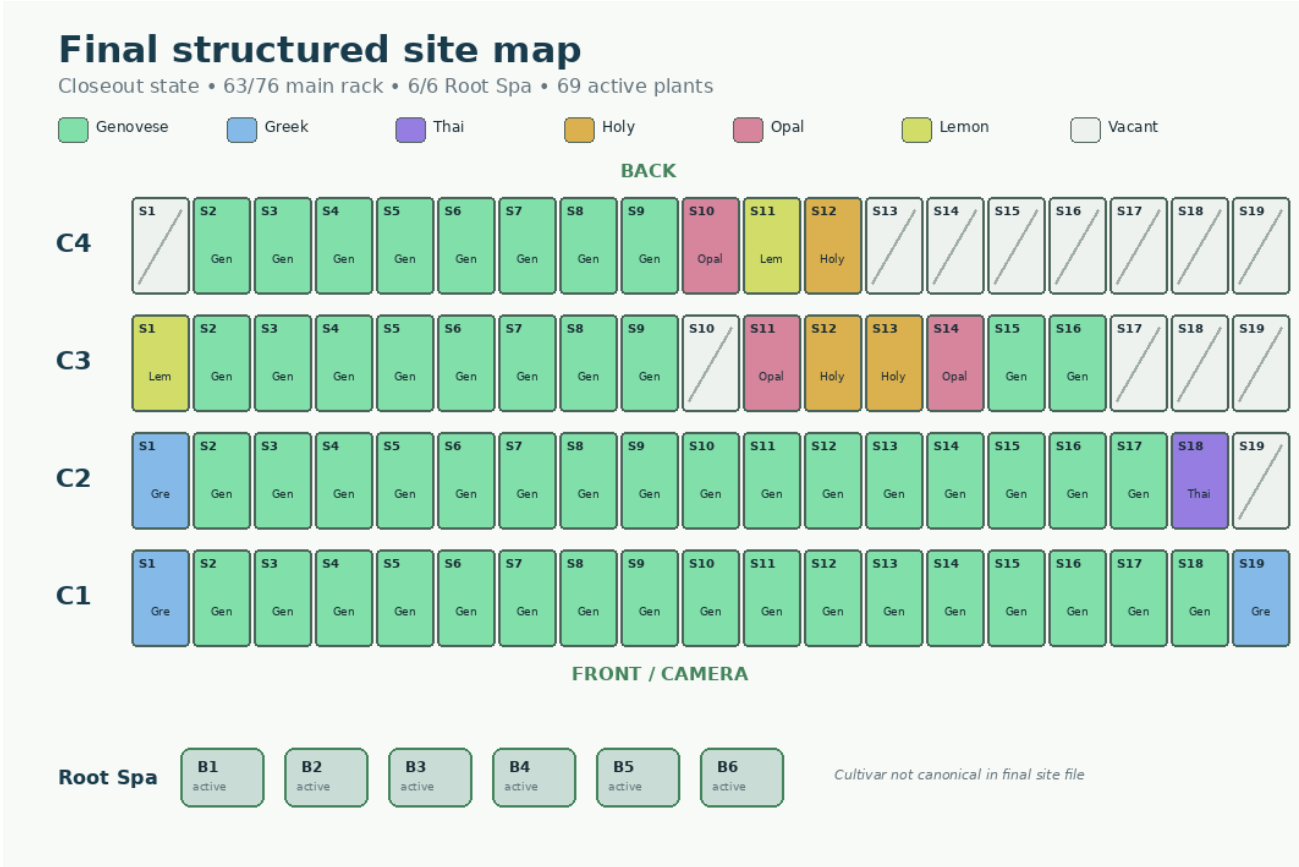


On June 12, measured canopy moved from 48.4% before the cut to 38.2% afterward. The next full recovery snapshot reached 49.8%, above the pre-harvest level, for an approximate 1.6-day recovery

interval. Canopy was not a direct biomass scale; it was an operational signal that could answer a useful question the spreadsheet alone could not: had the rack visually recovered from the cut?

4. Exact per-site vision was the wrong source of truth

The more ambitious idea was to infer individual plant state directly from the image. That became calibration-sensitive quickly. Plants overlap, lighting changes the apparent leaf color, glossy leaves blow out, the camera angle shifts, and one plant can occupy visual space that does not line up cleanly with a rectangular site ROI.



At closeout, exact occupancy still came from the structured site ledger—not from CV inference: 63/76 rack sites plus 6/6 Root Spa positions were active.

What the ledger solved

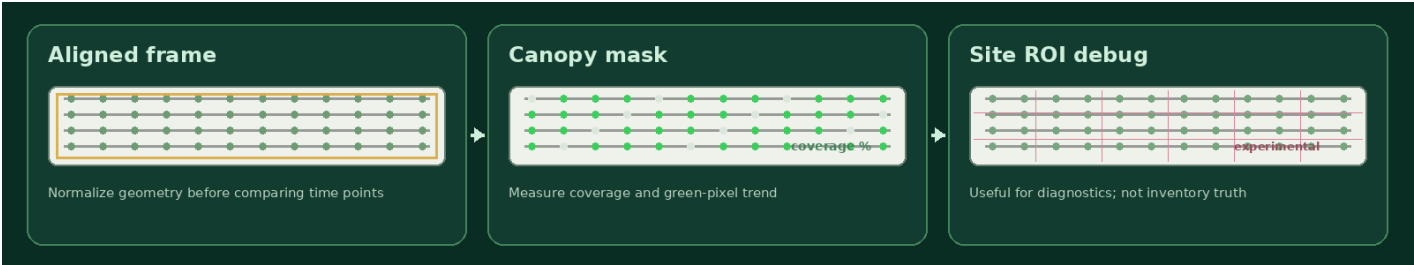
- Physical location stayed explicit: four channels × nineteen sites, plus the separate Root Spa.
- Cultivar identity survived camera ambiguity and canopy overlap.
- Transplants and movements could be recorded as events rather than reverse-engineered from pixels.
- Vacant sites remained meaningful even when neighboring foliage visually covered them.

Engineering lesson

Perception is allowed to be probabilistic. Inventory is not. A reliable system should separate the two instead of forcing a vision model to answer questions better handled by explicit state.

5. Quality controls mattered as much as the segmentation

The difficult part was not turning green pixels into a percentage. It was knowing whether two percentages were actually comparable. That pushed the project toward alignment checks, canonical crop definitions, confidence values, ignored-frame handling, camera epochs, and stale-sensor warnings.



The pipeline produced three different artifacts: geometry-normalized frames, whole-rack canopy measurements, and an explicitly experimental per-site diagnostic view.



Failure modes observed in practice

- Camera movement changes the geometry enough that trends across viewpoints must be treated as different calibration epochs.
- Grow-light glare and pale/glossy foliage can undercount “green” canopy even when the plants are healthy.
- A visually bad or lights-off frame should be ignored explicitly, not allowed to corrupt the trendline.
- Sensor freshness belongs in the UI. A number without data age can look authoritative long after it stopped updating.
- Image coverage ended before the farm experiment did, proving that the farm record could not depend on the camera pipeline.

By closeout, the software explicitly documented these limitations instead of hiding them. That made the dataset more useful: later analysis could distinguish measured state from experimental vision output.

6. The architecture became deterministic where it mattered

The project started with a camera-centric idea, but the final system behaved more like an event-sourced operational record with vision attached. That proved to be the more robust design.

Early instinct	What I would keep now
Infer individual plant state from the image	Use vision for whole-rack trend and anomalies
Treat the dashboard as the state	Store farm state independently; let the dashboard render it
Compare every frame to every other frame	Use camera/calibration epochs and comparable windows
Let a forecast update as new data arrives	Seal pre-harvest forecasts, then resolve against actual harvest
Add intelligence first	Add deterministic records and QA first

The useful abstraction

The camera is a sensor. The ledger is the farm. Harvest weights close the loop. The dashboard is the operator interface.

That separation made the final archive defensible enough to support the commercial case study later. The pattern generalizes beyond basil: when perception is uncertain, record real-world state explicitly and use CV to add context rather than overwrite it.

Forecasting became auditable, not self-correcting

Two sealed pre-harvest forecasts were resolved against actual weight. June 22: 246-410 g forecast, 241 g actual (just below range). June 30: 312-520 g forecast, 340 g actual (inside range). Both were low-confidence. The important improvement was procedural: forecasts were frozen before harvest and evaluated afterward instead of being rewritten with hindsight.

7. What I would carry forward

If I rebuilt the software for a managed restaurant or residential rack, I would make it smaller, more operational, and less interested in proving that computer vision is clever.

Fixed imaging geometry: A hard-mounted camera and explicit calibration epoch whenever the rack or camera moves.

Trend-first CV: Canopy growth, harvest recovery, blockage/anomaly cues, and visual proof of system state.

Deterministic inventory: Site map, crop plan, planting date, service history, and harvest records remain explicit.

Remote health: Water/leak status, pump state, sensor freshness, and a small number of actionable alerts.

Service workflow: Technician-facing checklist and swap-level components rather than a research dashboard.

Forecasting: Simple sealed forecasts tied to expected harvest windows and later resolved against actual weight.

What this project demonstrated

The result was not a production-grade vision product. It was a field prototype deployed against a real biological system long enough to expose the parts that are easy in a demo and hard in operation: calibration, ground truth, stale data, changing geometry, and the difference between a useful signal and a reliable fact.

Next commercial hypothesis

The companion farm study concluded that the rack itself may be more valuable as a managed living system than as a machine for producing commodity basil. If that direction is tested, the vision layer should support low-cost service and remote confidence—not become another thing the customer or technician has to babysit.

Companion document: "Can a Garage Become a Farm?" - commercial and physical-system case study.

Data and figures reconstructed from the frozen Garage Vision repository archive (closeout September 17, 2026) and the reviewed experiment workbook. The June 15 project report is retained only as a historical UI reference.